

STEP 10

Monitoring and data drift

Detect when the data the model sees in production no longer looks like the data it was trained on.

What is data drift?

A model learns patterns from its training data. If the real world changes, for example a heatwave, a new supplier, or a change in how staff record data, the inputs shift and predictions silently get worse. Drift monitoring compares new inputs with the training snapshot so the change is noticed *before* it causes damage.

How the check works

Feature type	Test	Effect size	Flagged as drift when
Numeric (4)	Kolmogorov-Smirnov	KS statistic	$p < 0.05$ and $KS \geq 0.10$
Categorical (4)	Chi-square	Total variation distance (TVD)	$p < 0.05$ and $TVD \geq 0.10$

Both conditions are required. With thousands of rows a statistical test flags even tiny, harmless differences, and the effect-size threshold makes sure the change is big enough to matter.

Run it

```
python -m foodwaste.drift --simulate normal      # fresh sample, same conditions -> no drift
python -m foodwaste.drift --simulate heatwave    # 70% hot days, storage ~6 C warmer -> drift
python -m foodwaste.drift --from-logs           # requests received by the API
python -m foodwaste.drift --current new.csv      # any CSV with the input columns
# add --fail-on-drift to exit with code 1 (for schedulers and CI)
```

Result of the heatwave simulation

Feature	Statistic	p-value	Drifted
Storage_Temperature	KS 0.385 (mean 10.6 → 16.5 °C)	2.3e-113	Yes
Weather	TVD 0.538 (Hot 20% → 74%)	1.2e-271	Yes
The other 6 features	≤ 0.034	> 0.27	No

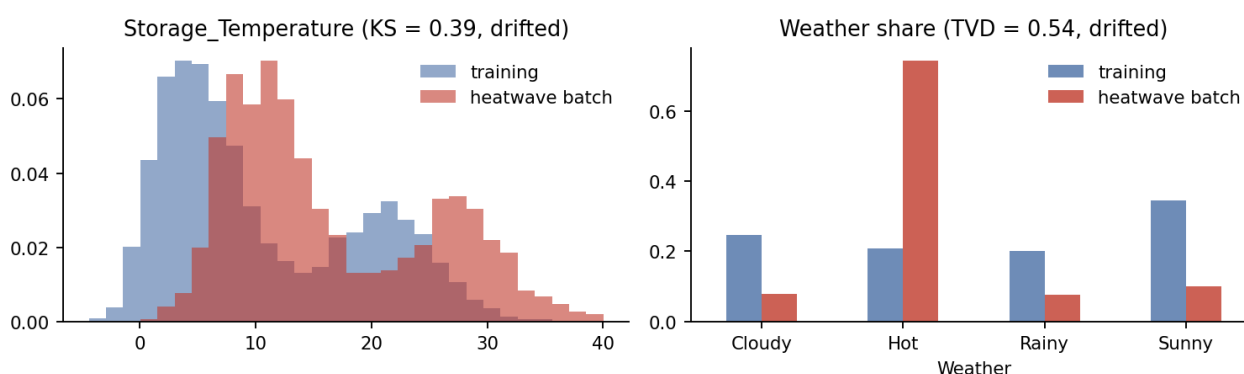


Figure 6 - The heatwave batch (red) against the training data (blue). These are the charts embedded in `reports/drift_heatwave.html`.

Reports

Each run writes `reports/drift_<name>.html` (status, table, charts of the drifted features) and a matching JSON file for machines. Open the HTML file in a browser.

Watch out

Drift tests need enough data. With fewer than 100 rows the script prints a warning: a handful of identical test requests will always look 'different' from 6,400 training rows.

Why it matters

Monitoring closes the MLOps loop: build → serve → monitor → retrain. In a real deployment the `--from-logs` check would run on a schedule, and `--fail-on-drift` would trigger retraining or an alert.